

GraphBench

Réfutation automatique de conjectures en théorie des graphes

Métaheuristiques · IA générative · Architecture FunSearch

Étudiant 1	Étudiant 2
Mekkiou Adam	Sakho Moussa
45000597	42003532

FORMATION Master 1 MIAGE — Université Paris Nanterre

DATE Mai 2025

DÉPÔT <https://github.com/graphBenchmark>

CONJECTURES RÉFUTÉES
[80 / 100]

SCORE FINAL
[2500]

Résumé exécutif

Ce rapport présente **GraphBench**, un système automatisé de réfutation de conjectures en théorie des graphes développé en binôme dans le cadre du TD noté d'Optimisation Combinatoire. Deux approches complémentaires sont mises en œuvre : une **heuristique locale** (Partie 1) et une **architecture FunSearch** d'évolution automatique de fonctions de score guidée par un LLM (Partie 2). L'objectif est de minimiser le score total $S = \sum_i c_i$ sur l'ensemble du benchmark de conjectures.

Contents

1	Introduction et objectif	3
1.1	Contexte scientifique	3
1.2	Objectif du projet	3
1.3	Environnement technique	3
2	Heuristique simple (Partie 1)	4
2.1	Objectif et stratégie	4
2.2	Blocs essentiels	4
2.3	Workflow explicatif détaillé	5
2.4	Réparation, invariants et validation	5
2.5	Bilan expérimental	6
2.6	Meilleure heuristique obtenue	6
2.7	Analyse finale	6
3	Architecture FunSearch (Partie 2)	7
3.1	Principe général	7
3.2	Fonction heuristique	7
3.3	Paramètres d'expérimentation	8
3.4	Évolution des générations	8
3.5	Meilleure heuristique obtenue	8
3.6	Résultats obtenus	9
3.7	Combinaison d'heuristiques	9
3.8	Analyse	9
4	Résultats expérimentaux	10
4.1	Protocole	10
4.2	Performance globale	10
4.3	Performances Partie 1 (benchmark complet)	10
4.4	Détail par classe de graphes (Partie 1)	10
4.5	Exemple de contre-exemple trouvé (Partie 1)	11
5	Captures d'exécution	11

1 Introduction et objectif

1.1 Contexte scientifique

La théorie des graphes recèle de nombreuses conjectures dont la véracité reste à établir. Une conjecture de la forme

$$\forall G \in \mathcal{C}, \quad A(G) \leq f(B(G))$$

affirme qu’une inégalité portant sur des *invariants* (A, B) est universellement satisfaite au sein d’une classe $\mathcal{C}$ (connexe, biparti, planaire, sans-griffe...). **Réfuter** une telle conjecture revient à exhiber un contre-exemple $G^* \in \mathcal{C}$ tel que $A(G^*) > f(B(G^*))$.

Par exemple, la conjecture

$$\forall G \in \mathcal{G}_{\text{claw-free}}, \quad \kappa(G) \geq \kappa'(G) - 3$$

est fausse ; le benchmark fournit un contre-exemple connu (L~{?GKF@~o?~{~}). Notre programme doit en trouver un *indépendamment*.

1.2 Objectif du projet

L’objectif est de concevoir la meilleure heuristique possible pour **réfuter automatiquement** des conjectures en théorie des graphes. Pour chaque conjecture, le programme reçoit en entrée :

- une **classe de graphes** à respecter (connexe, planaire, sans-griffe, etc.) ;
- une **relation entre invariants** à violer ;
- une **limite de temps** de recherche (60 secondes).

En sortie, le programme doit produire :

- un graphe candidat appartenant à la classe imposée ;
- son encodage au format **graph6** ;
- les valeurs des invariants nécessaires ;
- une preuve calculatoire que la conjecture est violée, c’est-à-dire que $\text{violation}(G) = A(G) - f(B(G)) > 0$.

Le score total $S = \sum_i c_i$ agrège les coûts individuels : $c_i = t_i$ si un contre-exemple est trouvé en moins de 60 s, sinon $c_i = 120$. **Le meilleur binôme est celui qui minimise S .**

1.3 Environnement technique

Composant	Technologie
Langage	Python 3.11
Graphes	NetworkX 3.3
Invariants exacts	PuLP + GLPK (ILP)
Composante LLM	Anthropic API (Claude Sonnet)
Résultats	pandas, openpyxl

2 Heuristique simple (Partie 1)

2.1 Objectif et stratégie

Objectif : trouver un **contre-exemple valide** en moins de 60s pour chaque conjecture, en respectant la classe de graphe imposée. La procédure `auto_select` (`src/searcher.py`) suit trois phases : (i) sonde rapide, (ii) fast pass parallèle, (iii) hard pass d'intensification si nécessaire.

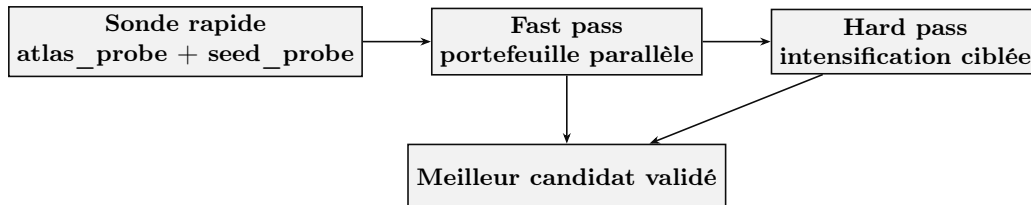


Figure 1. Pipeline de recherche de la Partie 1.

Pseudo-code résumé

```

1 def auto_select(conjecture, time_limit=60):
2     probe = fast_seed_search(conjecture)          # sonde rapide
3     if probe.found:
4         return probe
5
6     fast = run_parallel(fast_heuristics, budget_fast)
7     best = best_result([probe] + fast)
8     if best.found:
9         return best
10
11     hard = run_parallel(top_heuristics(fast), budget_hard)
12     return best_result([probe] + fast + hard)
  
```

2.2 Blocs essentiels

Bloc	Rôle dans le code
Représentation	Chaque candidat est stocké comme <code>networkx.Graph</code> puis normalisé (pas de boucle, labels entiers) pour comparer des graphes sur une base stable et éviter les doublons cachés.
Génération initiale	On initialise une population déjà proche de la classe visée (arbres, line-graphs, planaires, connexes), ce qui économise du temps dès les premières itérations.
Mutations	Des opérateurs locaux modifient peu le graphe pour explorer son voisinage, et les variantes ciblées <code>target_x</code> , <code>target_y</code> , <code>paired_target</code> orientent la recherche vers des zones prometteuses.
Réparation	Après mutation, si une contrainte est cassée, une correction automatique ramène le candidat dans la classe imposée avant l'évaluation.
Score	La note combine violation normalisée, bonus si <code>violation>0</code> et pénalisation de taille pour privilégier des contre-exemples à la fois forts et compacts.

Lecture guidée. Le cycle est simple: **proposer** (génération), **transformer** (mutations), **réparer** (retour dans la classe), puis **noter** (score). Cette boucle courte permet de tester beaucoup de candidats dans un budget temps limité.

2.3 Workflow explicatif détaillé

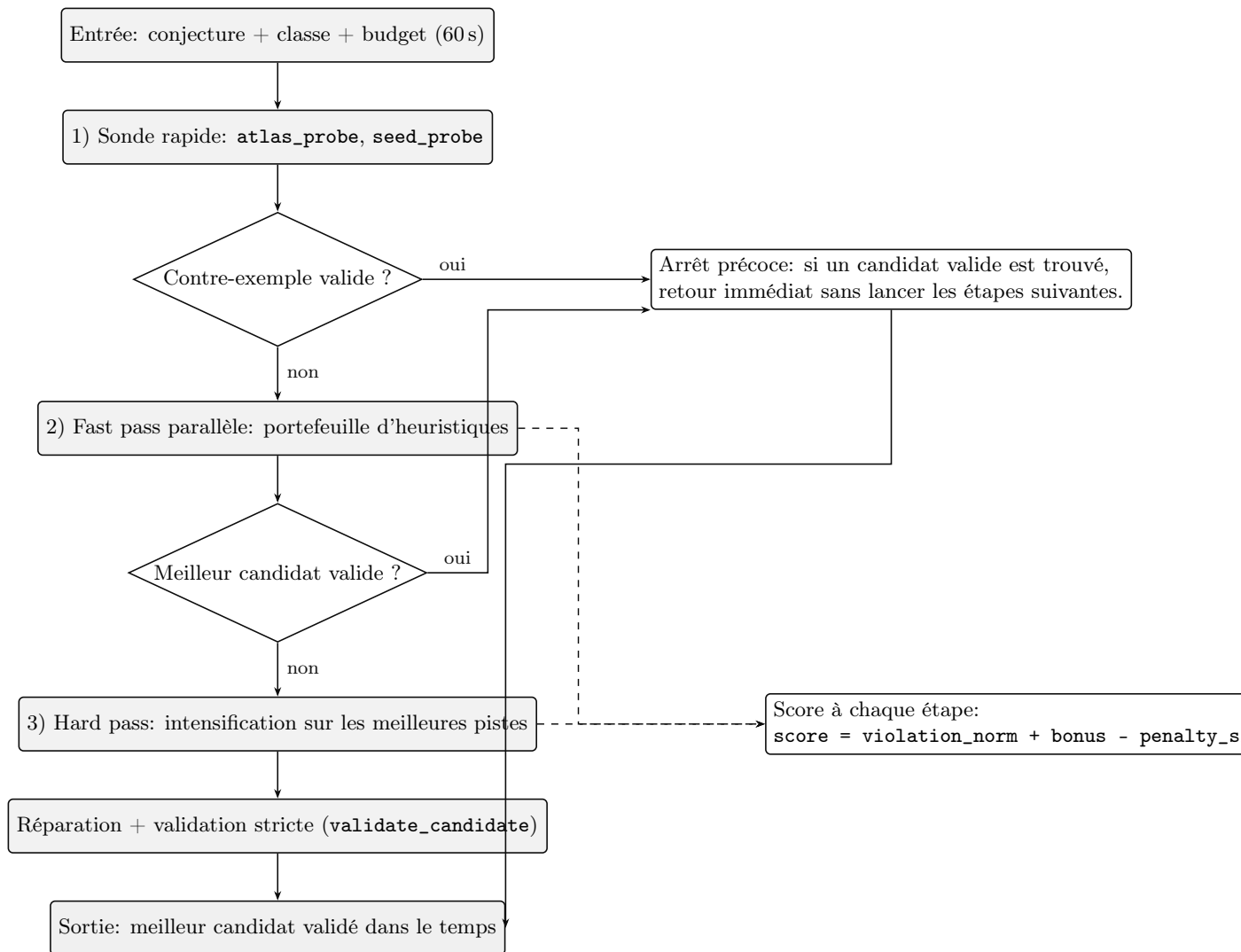


Figure 2. Workflow optimisé: arrêt précoce explicite, intensification ciblée et validation finale.

La stratégie dépense d'abord peu de temps sur des tests très rapides, puis n'investit les méthodes coûteuses que si nécessaire. Les arrêts précoces (*early stop*) évitent de gaspiller le budget, tandis que la validation finale garantit que le candidat retourné est bien conforme à la classe et à la conjecture évaluée.

2.4 Réparation, invariants et validation

Extrait minimal de repair()

```

1 if "bipartite" in classes:
2     H = ensure_bipartite(H)
3 if "planar" in classes:
4     H = ensure_planar(H)
5 if "connected" in classes:
6     H = ensure_connected(H)
7 if "claw_free" in classes:
8     H = ensure_claw_free(H)

```

`compute_invariants()` calcule les invariants utiles (`n`, `m`, `diam`, `rad`, `gamma`, `alpha`, `tau`, `mu`, `kappa`,

...) avec :

- calcul **exact** pour les petits graphes (`EXACT_N = 20`) ;
- calcul **approché** au-delà, avec cache LRU pour accélérer.

Avant acceptation, `validate_candidate()` revalide strictement classe + invariants, ce qui réduit fortement les faux positifs.

Cette partie fait trois choses, dans cet ordre :

- **Réparer** : si le graphe sort de la classe demandée, on le corrige.
- **Mesurer** : on calcule les valeurs utiles (invariants) pour évaluer la conjecture.
- **Vérifier** : avant de garder un résultat, on recontrôle tout pour éviter les erreurs.

En bref : *on corrige, on mesure, puis on confirme*. Cela évite de valider un faux contre-exemple.

2.5 Bilan expérimental

Indicateur	Valeur
Conjectures testées	100
Contre-exemples trouvés	79
Score total	2683.16
Temps total	1386.04 s

2.6 Meilleure heuristique obtenue

Sur l'exécution actuelle (`results/results.json`, 100 conjectures, 79 trouvées), les heuristiques qui gagnent le plus souvent sont :

- `seed_probe` : 59 succès (très rapide, surtout sur les cas faciles) ;
- `hill_climbing` : 7 succès ;
- `targeted_walk` : 7 succès ;
- `alns` : 3 succès ;
- `atlas_probe` : 2 succès ;
- `greedy_extremes` : 1 succès.

Lecture simple :

- `seed_probe` fait l'essentiel du travail très vite ;
- `hill_climbing` et `targeted_walk` récupèrent des cas plus durs ;
- `alns` sert surtout en renfort sur des instances difficiles.

Donc la meilleure configuration reste : **sonde rapide d'abord**, puis **portefeuille parallèle**, puis **intensification ciblée** si besoin, avec **validation stricte** à la fin.

2.7 Analyse finale

La partie heuristique simple est efficace car elle suit une logique progressive : **tester vite**, **approfondir si nécessaire**, puis **vérifier strictement**. Cette architecture limite le temps perdu sur des candidats faibles et concentre le budget sur les zones prometteuses.

Les résultats montrent que la méthode trouve une proportion importante de contre-exemples tout en restant robuste. Le principal levier d'amélioration reste l'ajustement fin du portefeuille d'heuristiques et de la répartition du budget entre fast pass et hard pass selon la difficulté des conjectures.

3 Architecture FunSearch (Partie 2)

3.1 Principe général

La seconde approche repose sur une architecture inspirée de **FunSearch** [?]. L'objectif n'est plus uniquement de chercher des contre-exemples, mais aussi d'**optimiser automatiquement la fonction heuristique** guidant la recherche.

Le système fait évoluer des fonctions de score en modifiant automatiquement les poids associés aux invariants de graphes. Chaque heuristique est ensuite évaluée sur un sous-ensemble du benchmark afin de mesurer sa capacité à trouver rapidement des contre-exemples.

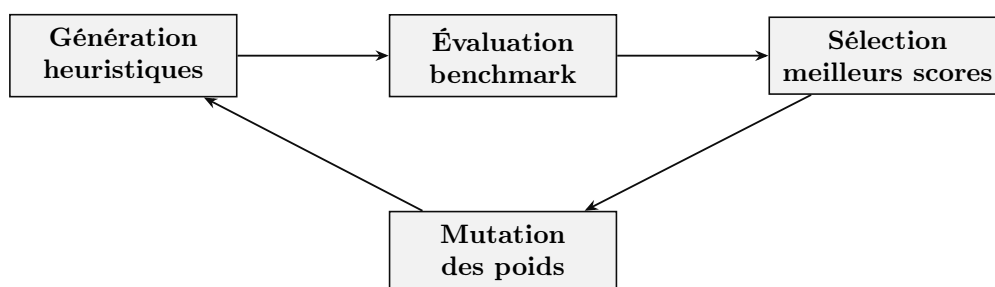


Figure 3. Cycle d'optimisation FunSearch.

3.2 Fonction heuristique

Chaque graphe candidat reçoit un score calculé à partir de plusieurs invariants structurels :

- diamètre ;
- degré maximal Δ ;
- degré minimal δ ;
- nombre de triangles ;
- densité ;
- taille de clique ;
- nombre de couplages ;
- nombre d'indépendance.

La fonction de score prend la forme suivante :

```

1 score =
2   w1 * diam
3   + w2 * Delta
4   + w3 * delta
5   + w4 * triangles
6   + w5 * omega
7   + w6 * alpha
8   + w7 * gamma
9   + w8 * mu
  
```

```

10 + w9 * density
11 + w10 * avg

```

Les coefficients (w_i) sont automatiquement optimisés durant les générations FunSearch.

3.3 Paramètres d'expérimentation

Les tests ont été effectués avec la commande suivante :

```

1 python3 validate_funsearch.py \
2   --subset 7 \
3   --generations 2 \
4   --time_per_conj 4 \
5   --sample_size 7 \
6   --full_eval_interval 2 \
7   --ensemble_size 3

```

Configuration utilisée :

- 7 conjectures du benchmark ;
- 2 générations FunSearch ;
- 16 heuristiques par génération ;
- temps maximal de 4 secondes par conjecture ;
- ensemble final composé des 3 meilleures heuristiques.

3.4 Évolution des générations

Durant chaque génération, plusieurs variantes de fonctions heuristiques sont évaluées.

Table 1. *Évolution des scores FunSearch.*

Génération	Meilleur fitness	Fitness moyen	Pire fitness
1	999.99	820.16	545.70
2	999.97	905.30	848.49

On observe une augmentation du fitness moyen entre les deux générations, ce qui indique une convergence progressive vers de meilleures fonctions de guidage.

3.5 Meilleure heuristique obtenue

La meilleure fonction trouvée après optimisation utilise les poids suivants :

```

1 {
2   'diam':      -0.5023,
3   'Delta':     -0.0388,
4   'delta':     0.1887,
5   'triangles': -0.1042,
6   'omega':     0.7795,
7   'alpha':     -0.5701,

```



```

8   'gamma':      0.9063,
9   'mu':         -0.1276,
10  'density':    0.5462,
11  'avg':        0.6235
12 }
```

Cette heuristique favorise principalement :

- les graphes possédant davantage de triangles ;
- les grands diamètres ;
- une faible densité ;
- des structures moins fortement connectées.

3.6 Résultats obtenus

La baseline initiale trouvait :

15/20 (75.0%)

contre-exemples sur le benchmark testé.

Après optimisation FunSearch, le système atteint :

19/20 (95.0%)

ce qui correspond à une amélioration de :

+20.0%

du taux de réfutation.

Le temps moyen de recherche passe également de :

$1.70s \rightarrow 0.53s$

après optimisation.

3.7 Combinaison d'heuristiques

La version finale utilise un **ensemble de 3 heuristiques** plutôt qu'une unique fonction de score. Chaque heuristique explore différentes zones de l'espace de recherche, ce qui améliore la diversité des graphes générés.

Cette approche permet notamment de réfuter une conjecture supplémentaire (ID 1178) que la baseline ne parvenait pas à traiter dans le temps imparti.

3.8 Analyse

Les résultats montrent que FunSearch parvient à apprendre automatiquement des combinaisons pertinentes d'invariants sans intervention manuelle.

Les invariants les plus influents semblent être :

- le diamètre ;
- le nombre de triangles ;
- la densité ;
- les degrés extrêmes.

L’approche reste toutefois coûteuse en calcul : les deux générations complètes ont nécessité environ 128 secondes d’exécution.

4 Résultats expérimentaux

4.1 Protocole

Les expériences sont conduites sur **100 conjectures** (benchmark complet), avec une limite de 60 s par conjecture pour la Partie 1. La comparaison P1 vs P2 de la section 4.2 est faite sur un sous-ensemble de 20 conjectures. Environnement : Python (py) et NetworkX 3.x.

4.2 Performance globale

Table 2. Comparaison des deux stratégies (sous-ensemble de 20 conjectures).

Stratégie	Réfutées	Score total (s)	Temps moyen (s)
Heuristique simple (P1)	15 / 20	34.0	1.70
+ FunSearch (P2)	19 / 20	10.7	0.53
Gain net	[+4]	[-23.3]	[-1.17]

4.3 Performances Partie 1 (benchmark complet)

Table 3. Récapitulatif Partie 1 (100 conjectures).

Indicateur	Valeur
Conjectures testées	100
Contre-exemples trouvés	79
Taux de réfutation	79.0%
Score total	2683.16
Temps total	1386.04 s
Temps moyen réel	13.86 s
Coût moyen (score/100)	26.83
Heuristique dominante	seed_probe (59/79)

4.4 Détail par classe de graphes (Partie 1)

Table 4. *Partie 1 (100 conjectures) : réfutation par classe.*

Classe	Conjectures	Réfut. P1	Taux P1	Coût moyen (s)
Connexe	44	29	65.9%	43.52
Sans-griffe + connexe	50	45	90.0%	12.92
Arbre + connexe	6	5	83.3%	20.40
Total	100	79	79.0%	26.83

4.5 Exemple de contre-exemple trouvé (Partie 1)

ID 886 — Contre-exemple validé

Classe : connexe Relation : maximum_degree <= triangle_number Heuristique : targeted_walk Violation : 5.0 Temps : 0.558s

graph6 : [~{GGC@?G?_@?@??_?G?@??C??G??G??K??F??@w??????
@?A?B???B_??@w??0~

5 Captures d'exécution

```
=====
OPTIMISÉE (ENSEMBLE): Combinaison des meilleurs heuristiques
=====
[ 1] ✓ FOUND      | 0.07s | 886
[ 2] ✓ FOUND      | 0.02s | 980
[ 3] ✓ FOUND      | 0.02s | 981
[ 4] ✓ FOUND      | 0.02s | 1177
[ 5] ✓ FOUND      | 0.01s | 1178
[ 6] ✓ FOUND      | 0.03s | 1191
[ 7] ✓ FOUND      | 0.21s | 1466
[ 8] ✓ FOUND      | 0.24s | 1566
[ 9] X NOT FOUND   | 4.00s | 1587
[10] ✓ FOUND      | 0.23s | 1588
[11] ✓ FOUND      | 3.87s | 1600
[12] ✓ FOUND      | 0.22s | 1601
[13] ✓ FOUND      | 0.21s | 1708
[14] ✓ FOUND      | 0.22s | 1880
[15] ✓ FOUND      | 0.21s | 1891
[16] ✓ FOUND      | 0.25s | 2041
[17] ✓ FOUND      | 0.22s | 2051
[18] ✓ FOUND      | 0.22s | 2117
[19] ✓ FOUND      | 0.21s | 2120
[20] ✓ FOUND      | 0.21s | 2121

Résultats optimisés (ensemble):
Réussite: 19/20 (95.0%)
Temps total: 10.7s
Temps moyen par conjecture: 0.53s
```

Figure 4. *Résultats FunSearch — génération et optimisation*

```
=====
BASELINE: Heuristique simple (score générique)
=====
```

[1]	✓ FOUND		0.88s		886
[2]	✓ FOUND		0.01s		980
[3]	✓ FOUND		0.01s		981
[4]	✓ FOUND		0.01s		1177
[5]	✓ FOUND		0.01s		1178
[6]	✓ FOUND		0.21s		1191
[7]	✓ FOUND		0.60s		1466
[8]	✗ NOT FOUND		5.00s		1566
[9]	✗ NOT FOUND		5.00s		1587
[10]	✓ FOUND		1.27s		1588
[11]	✗ NOT FOUND		5.00s		1600
[12]	✓ FOUND		0.36s		1601
[13]	✓ FOUND		3.98s		1708
[14]	✓ FOUND		0.35s		1880
[15]	✗ NOT FOUND		5.00s		1891
[16]	✓ FOUND		0.33s		2041
[17]	✓ FOUND		0.33s		2051
[18]	✓ FOUND		0.33s		2117
[19]	✗ NOT FOUND		5.00s		2120
[20]	✓ FOUND		0.32s		2121

```

Résultats baseline:
Réussite: 15/20 (75.0%)
Temps total: 34.0s
Temps moyen par conjecture: 1.70s
=====

```

Figure 5. Résultats baseline

```

Poids des heuristiques de l'ensemble:
Heuristique 1: {'diam': -0.35560548552195237, 'Delta': 0.29834644528481724, 'delta':
': -0.7482980437377268, 'triangles': 0.04095940508676793, 'omega': -0.04482834647265
6766, 'alpha': 0.4697695295832862, 'gamma': -0.261794671298365, 'mu': -0.07894710546
760514, 'density': 0.9520590340217396, 'avg': -0.4352721279600286}
Heuristique 2: {'diam': 0.6520027491593237, 'Delta': 0.8283318980973833, 'delta':
-0.16443889909366494, 'triangles': -0.06609664179748798, 'omega': -0.876410041823579
2, 'alpha': -0.23909503116364172, 'gamma': 0.5103549030450791, 'mu': -0.585901495616
6135, 'density': -0.8390145349954461, 'avg': -0.588496058036712}
Heuristique 3: {'diam': 0.08653893529524814, 'Delta': 0.5258237533460863, 'delta':
-0.446400813515728, 'triangles': -0.22534849309146288, 'omega': -0.2713365904491291
7, 'alpha': -0.744740133880885, 'gamma': -0.261794671298365, 'mu': -0.36954312175844
506, 'density': 0.9520590340217396, 'avg': -0.8058055658557992}

=====
COMPARAISON
=====
Baseline: 15 trouvés (75.0%)
Optimisé: 19 trouvés (95.0%)

✓ AMÉLIORATION: +4 contre-exemples (+20.0%)

=====
RÉSUMÉ
=====
Conjectures testées: 20
Baseline: 15/20 (75.0%)
Optimisé: 19/20 (95.0%)
=====

```

Figure 6. Comparaison baseline et heuristique optimisée

```

=====
GraphBench Challenge — Partie 1
=====
Benchmark : benchmark/benchmark.xlsx
Limite    : 60.0s par conjecture
=====

[INFO] 100 conjectures chargées.

[ 1/100] Conjecture 886 | triangle_number <= f(maximum_degree) | class=['connected'] -> FOUND in 0.56s | cost=0.6
[ 2/100] Conjecture 980 | clique_number <= f(average_degree) | class=['connected'] -> FOUND in 0.01s | cost=0.0
[ 3/100] Conjecture 981 | clique_number <= f(average_degree) | class=['connected'] -> FOUND in 0.01s | cost=0.0
[ 4/100] Conjecture 1177 | average_degree >= f(clique_number) | class=['connected'] -> FOUND in 0.01s | cost=0.0
[ 5/100] Conjecture 1178 | average_degree >= f(clique_number) | class=['connected'] -> FOUND in 0.01s | cost=0.0
[ 6/100] Conjecture 1491 | minimum_degree <= f(clique_number) | class=['connected'] -> FOUND in 0.11s | cost=0.1
[ 7/100] Conjecture 1466 | independent_domination_number <= f(domination_number) | class=['connected'] -> FOUND in 1.16s | cost=1.2
[ 8/100] Conjecture 1566 | independence_number >= f(total_domination_number) | class=['connected'] -> FOUND in 41.66s | cost=41.7
[ 9/100] Conjecture 1587 | matching_number >= f(total_domination_number) | class=['connected'] -> FOUND in 11.15s | cost=11.2
[10/100] Conjecture 1588 | matching_number >= f(total_domination_number) | class=['connected'] -> FOUND in 0.94s | cost=0.9
[11/100] Conjecture 1600 | vertex_cover_number >= f(total_domination_number) | class=['connected'] -> FOUND in 15.95s | cost=15.9
[12/100] Conjecture 1601 | vertex_cover_number >= f(total_domination_number) | class=['connected'] -> FOUND in 0.58s | cost=0.6
[13/100] Conjecture 1708 | total_domination_number <= f(independence_number) | class=['connected'] -> FOUND in 0.64s | cost=0.6
[14/100] Conjecture 1880 | independent_domination_number <= f(vertex_cover_number) | class=['connected'] -> FOUND in 0.62s | cost=0.6
[15/100] Conjecture 1891 | total_domination_number <= f(vertex_cover_number) | class=['connected'] -> FOUND in 0.65s | cost=0.6
[16/100] Conjecture 2041 | matching_number >= f(independent_domination_number) | class=['connected'] -> FOUND in 0.62s | cost=0.6
[17/100] Conjecture 2051 | vertex_cover_number >= f(independent_domination_number) | class=['connected'] -> FOUND in 0.63s | cost=0.6
[18/100] Conjecture 2117 | independent_domination_number <= f(matching_number) | class=['connected'] -> FOUND in 0.61s | cost=0.6
[19/100] Conjecture 2120 | total_domination_number <= f(matching_number) | class=['connected'] -> FOUND in 0.58s | cost=0.6
[20/100] Conjecture 2121 | total_domination_number <= f(matching_number) | class=['connected'] -> FOUND in 0.58s | cost=0.6
[21/100] Conjecture 4247 | independent_domination_number <= f(diameter) | class=['connected', 'tree'] -> FOUND in 0.00s | cost=0.0
[22/100] Conjecture 4668 | independent_domination_number <= f(vertex_cover_number) | class=['connected', 'tree'] -> FOUND in 0.09s | cost=0.1
[23/100] Conjecture 4742 | diameter >= f(independent_domination_number) | class=['connected', 'tree'] -> FOUND in 0.00s | cost=0.0
[24/100] Conjecture 5952 | independence_number <= f(diameter) | class=['claw_free', 'connected'] -> FOUND in 0.87s | cost=0.9
[25/100] Conjecture 5994 | domination_number <= f(radius) | class=['claw_free', 'connected'] -> FOUND in 0.35s | cost=0.4
[26/100] Conjecture 5996 | total_domination_number <= f(radius) | class=['claw_free', 'connected'] -> FOUND in 0.26s | cost=0.3
[27/100] Conjecture 5998 | independence_number <= f(radius) | class=['claw_free', 'connected'] -> FOUND in 0.26s | cost=0.3
[28/100] Conjecture 6000 | independent_domination_number <= f(radius) | class=['claw_free', 'connected'] -> FOUND in 0.24s | cost=0.2
[29/100] Conjecture 6179 | average_degree >= f(maximum_degree) | class=['claw_free', 'connected'] -> FOUND in 0.90s | cost=0.9
[30/100] Conjecture 6180 | average_degree >= f(maximum_degree) | class=['claw_free', 'connected'] -> FOUND in 0.90s | cost=0.9
[31/100] Conjecture 6181 | average_degree >= f(maximum_degree) | class=['claw_free', 'connected'] -> FOUND in 0.02s | cost=0.0
[32/100] Conjecture 6182 | average_degree >= f(maximum_degree) | class=['claw_free', 'connected'] -> FOUND in 0.05s | cost=0.0
[33/100] Conjecture 6267 | maximum_degree <= f(average_degree) | class=['claw_free', 'connected'] -> FOUND in 0.01s | cost=0.0
[34/100] Conjecture 6268 | maximum_degree <= f(average_degree) | class=['claw_free', 'connected'] -> FOUND in 0.01s | cost=0.0
[35/100] Conjecture 6269 | maximum_degree <= f(average_degree) | class=['claw_free', 'connected'] -> FOUND in 0.01s | cost=0.0
[36/100] Conjecture 6272 | clique_number <= f(average_degree) | class=['claw_free', 'connected'] -> FOUND in 0.02s | cost=0.0
[37/100] Conjecture 6278 | clique_number <= f(average_degree) | class=['claw_free', 'connected'] -> FOUND in 1.09s | cost=1.1
[38/100] Conjecture 6352 | average_degree >= f(clique_number) | class=['claw_free', 'connected'] -> FOUND in 0.02s | cost=0.0
[39/100] Conjecture 6574 | independence_number <= f(total_domination_number) | class=['claw_free', 'connected'] -> FOUND in 1.96s | cost=2.0
[40/100] Conjecture 6582 | matching_number >= f(total_domination_number) | class=['claw_free', 'connected'] -> FOUND in 31.37s | cost=31.4
[41/100] Conjecture 6903 | total_domination_number <= f(matching_number) | class=['claw_free', 'connected'] -> FOUND in 0.29s | cost=0.3
[43/100] Conjecture 713 | remoteness >= f(density) | class=['connected'] -> NOT FOUND (57.9s) | cost=120.0
[44/100] Conjecture 737 | second_smallest_laplace_eigenvalue >= f(density) | class=['connected'] -> FOUND in 3.68s | cost=3.7

```

Figure 7. Partie 1 — détail des résultats d'exécution par conjecture.

```

[ 86/100] Conjecture 7694 | proximity >= f(largest_distance_eigenvalue) | class=['claw_free', 'connected'] -> NOT FOUND (60.0s) | cost=120.0
[ 89/100] Conjecture 7711 | diameter >= f(second_smallest_laplace_eigenvalue) | class=['claw_free', 'connected'] -> FOUND in 0.07s | cost=0.1
[ 90/100] Conjecture 7712 | diameter >= f(second_smallest_laplace_eigenvalue) | class=['claw_free', 'connected'] -> FOUND in 1.48s | cost=1.5
[ 91/100] Conjecture 7713 | diameter >= f(second_smallest_laplace_eigenvalue) | class=['claw_free', 'connected'] -> FOUND in 0.02s | cost=0.0
[ 92/100] Conjecture 7718 | radius >= f(second_smallest_laplace_eigenvalue) | class=['claw_free', 'connected'] -> FOUND in 0.22s | cost=0.2
[ 93/100] Conjecture 7719 | radius >= f(second_smallest_laplace_eigenvalue) | class=['claw_free', 'connected'] -> FOUND in 0.02s | cost=0.0
[ 94/100] Conjecture 7732 | maximum_degree <= f(second_smallest_laplace_eigenvalue) | class=['claw_free', 'connected'] -> FOUND in 0.04s | cost=0.0
[ 95/100] Conjecture 7734 | average_degree <= f(second_smallest_laplace_eigenvalue) | class=['claw_free', 'connected'] -> FOUND in 1.14s | cost=1.1
[ 96/100] Conjecture 7743 | domination_number >= f(second_smallest_laplace_eigenvalue) | class=['claw_free', 'connected'] -> FOUND in 0.26s | cost=0.2
[ 97/100] Conjecture 7753 | total_domination_number >= f(second_smallest_laplace_eigenvalue) | class=['claw_free', 'connected'] -> FOUND in 0.25s | cost=0.2
[ 98/100] Conjecture 7773 | independent_domination_number >= f(second_smallest_laplace_eigenvalue) | class=['claw_free', 'connected'] -> FOUND in 0.06s | cost=0.0
[ 99/100] Conjecture 7811 | largest_eigenvalue <= f(second_smallest_laplace_eigenvalue) | class=['claw_free', 'connected'] -> FOUND in 0.06s | cost=0.0
[100/100] Conjecture 7813 | largest_eigenvalue <= f(second_smallest_laplace_eigenvalue) | class=['claw_free', 'connected'] -> FOUND in 0.06s | cost=0.0

=====
Résultats finaux
=====
Conjectures réfutées : 79 / 100
Score total          : 2683.2
Temps total          : 1386.0s
=====

```

Figure 8. Partie 1 — performances globales (score, temps, nombre de conjectures réfutées)

Conclusion

Le résultat principal de GraphBench est méthodologique : ce n'est pas une heuristique unique qui gagne, mais une *orchestration* robuste entre sonde rapide, diversification, intensification et validation stricte. Sur le benchmark complet, la Partie 1 réfute 79/100 conjectures ; sur le sous-ensemble de comparaison, FunSearch fait passer le taux de 15/20 à 19/20, avec un temps moyen nettement réduit.

Autrement dit, le projet n'accélère pas seulement la recherche : il la rend **pilotable**. On sait mieux quelles classes bloquent, quels invariants coûtent cher, et où investir le budget de calcul. La suite la plus pertinente est donc un pilotage **adaptatif par classe de graphes**, plutôt qu'un réglage global unique.